第5章 线程级并行（多处理机）

5.1 多处理机基础与MIMD并行机结构

5.1.1 计算机构成两大核心基本要素

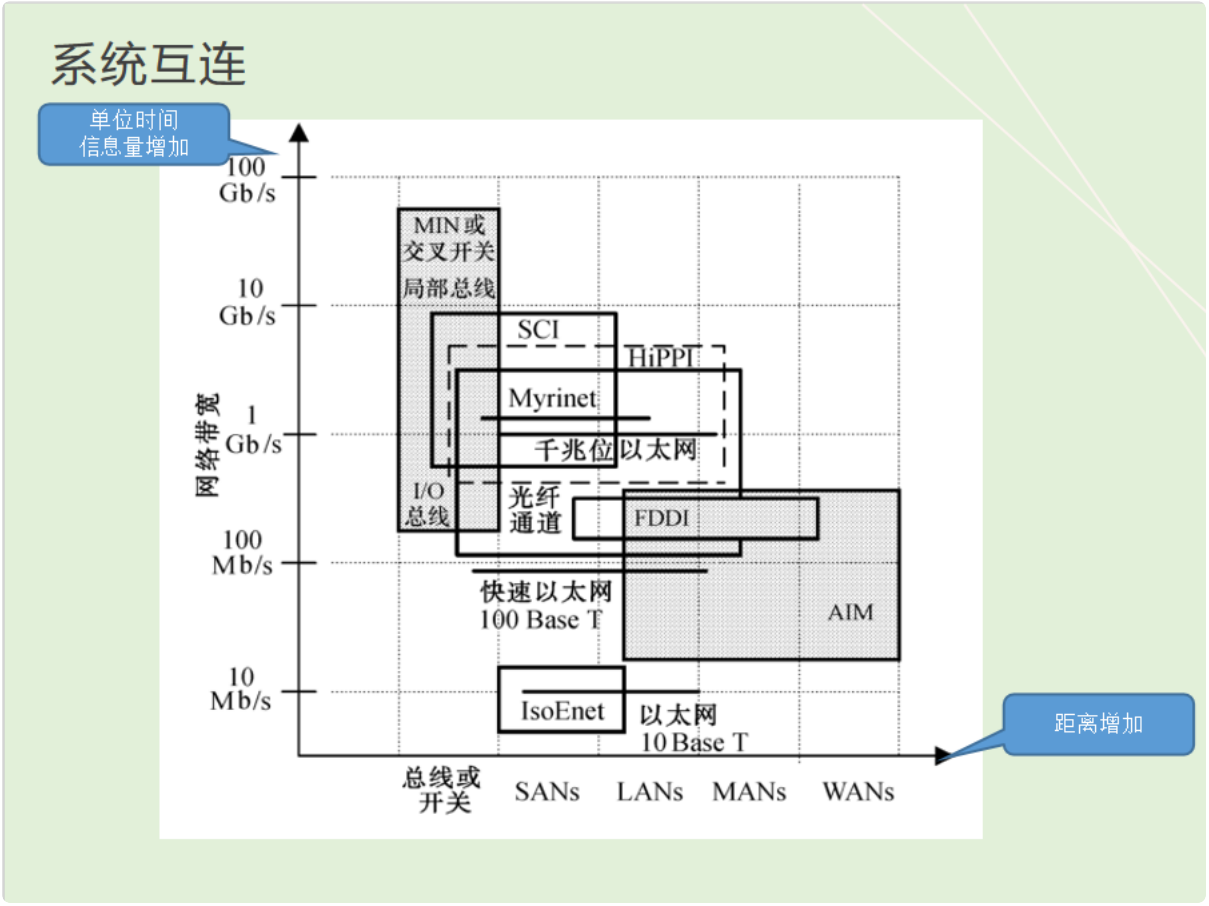
多处理机系统的所有硬件架构，均可拆解为**节点（点）、互连网络** 两大基础要素，是理解并行机架构的核心前提：

节点（点）：所有具备计算、存储、交互能力的硬件单元，层级覆盖范围极广。微观包含CPU内部寄存器、ALU算术逻辑单元、控制器；宏观包含存储模块、外设接口、多处理机独立计算节点。所有节点均可独立参与运算或数据存储。

互连网络：按照固定拓扑结构和传输控制协议，将所有独立节点连接起来的通信架构，核心作用是实现节点间的数据交互、指令传输，是并行计算的通信基础。

5.1.2 系统互连网络层级与特性

互连网络根据传输带宽、传输距离、应用场景分为多个层级，核心规律： **传输距离越近，能实现的带宽越高；传输距离越远，带宽上限越低**



5.1.3 互连网络核心分类（静态+动态）

1. 静态互连网络

定义：处理单元之间为**固定物理连接**，程序运行全过程中，节点间点到点的链路不会发生任何改变，拓扑结构永久固定。特点是结构稳定、通信延迟固定，灵活性差。

2. 动态互连网络

定义：由可编程开关单元组成，可根据应用程序的并行计算需求，**动态调整节点连接组态**，按需建立通信链路。核心代表：交叉开关网络、多级互连网络（MIN），灵活性高、适配性强，是现代多处理机主流互连方式。

5.1.4 五大MIMD并行机结构模型

MIMD（多指令多数据流）是多处理机核心架构，根据硬件组成、存储方式、互连模式分为五类，为期末核心考点，需区分架构特点、优缺点、代表机型及现状。

1. PVP 并行向量处理机

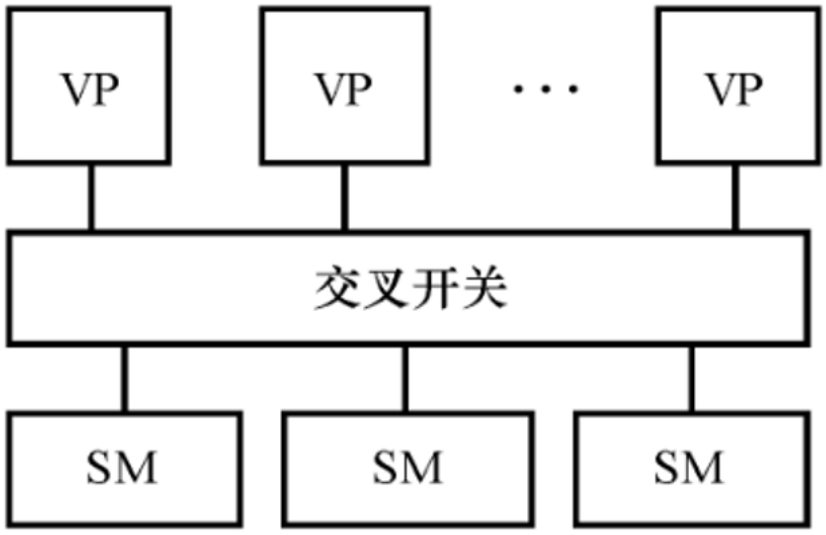
核心架构：系统搭载少量**定制高性能向量处理器（VP）**，单处理器算力极强（≥1Gflops），专为大规模科学计算设计。

存储与互连：存储器以超高带宽向处理器传输数据，所有向量处理器与共享存储模块通过**高带宽交叉开关**互连。

特殊设计：不使用传统高速缓存，依靠大量向量寄存器、指令缓冲器实现高速数据吞吐，适配批量向量运算。

代表机型：Cray90（1991）、NEC SX-4（1994）、中国银河1号（1983）

发展现状：独立PVP大型机已淘汰，但**向量并行处理核心思想**完全沿用，是现代CPU、GPU、AI加速器、高性能计算的底层核心技术。



2. SMP 对称多处理机

核心架构：采用通用商用微处理器（自带片上/外置高速缓存），所有处理器地位完全对等，通过高速总线或交叉开关连接共享存储器、I/O设备。

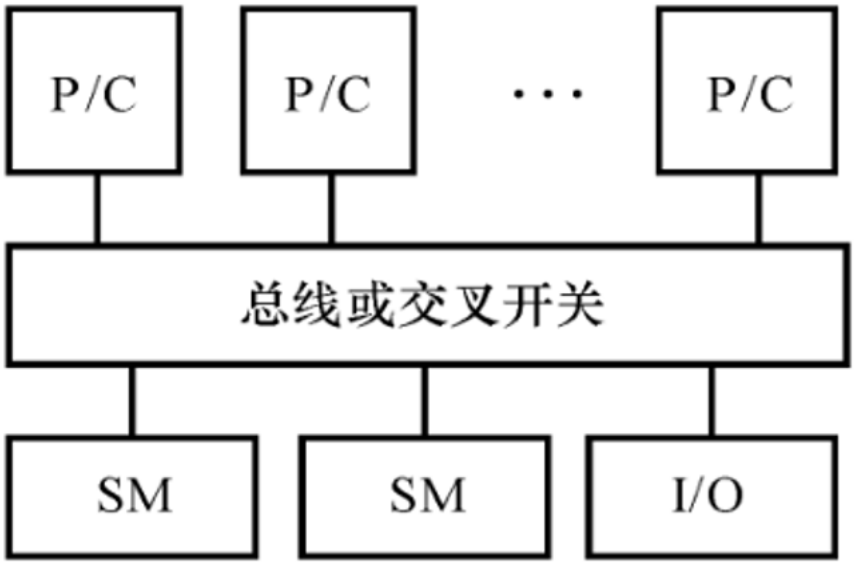
核心特性——对称性：所有处理器可**等同延迟、等同权限**访问共享内存、I/O设备、操作系统服务，是高并行度的核心基础。

优缺点：架构简单、编程友好、商务适配性强；但扩展性极差，处理器数量上限≤64个，总线与交叉开关硬件固定，无法后期扩容。

应用场景：商务计算、数据库、在线事务处理、数据仓库

代表机型：IBM R50、SGI Power Challenge、DEC Alpha 8400、中国曙光1号（首款国产SMP架构高性能计算机）

发展现状：独立大型SMP系统绝迹，现代CPU多核架构（16核、64核）本质是微型SMP系统。



3. DSM 分布式共享存储多处理机

核心架构：物理分布式存储，逻辑统一地址空间。各计算节点拥有独立局部物理内存，但通过硬件+软件封装，对用户呈现单一共享内存编程空间。

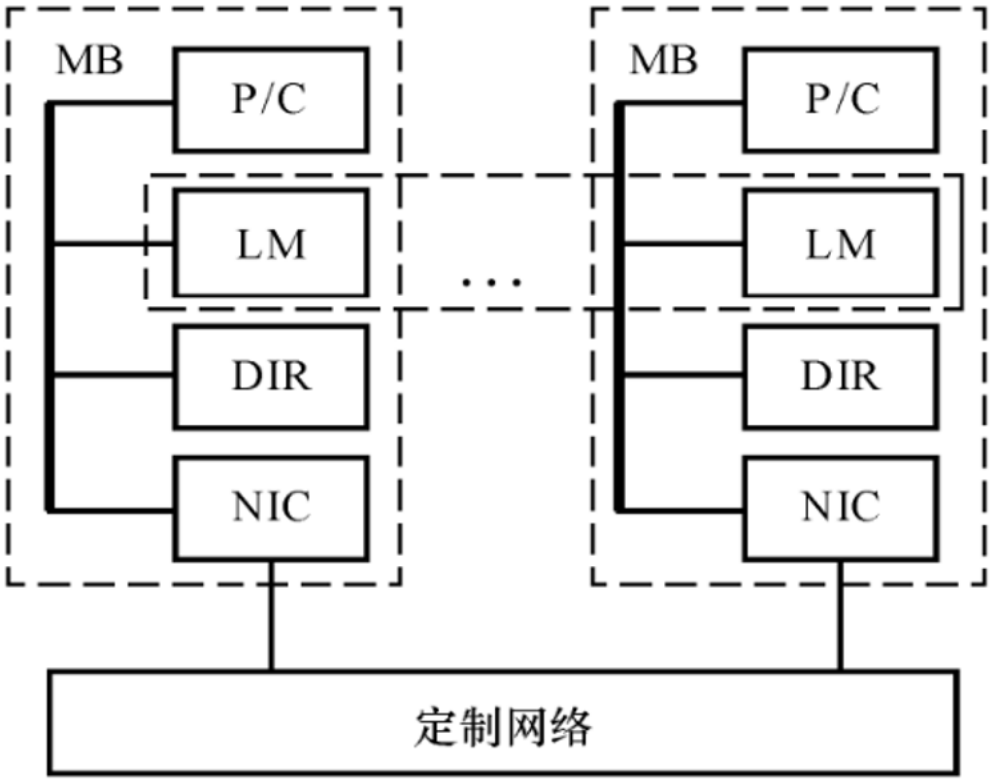
核心机制：增设高速缓存目录（DIR），专门维护分布式节点间的高速缓存一致性，解决多节点数据同步问题。

核心优势：对比MPP集群，编程模型简单，无需复杂消息传递，兼顾分布式扩展性与共享存储的易用性。

分类与代表：

软件DSM：中科院JIAJIA系统，广泛应用于曙光集群平台

硬件DSM：SGI Origin系列，基于NUMAlink高速互连，支持512+处理器，峰值万亿次算力



4. MPP 大规模并行处理机

核心定位：超大型高性能并行计算机，是全球TOP500超算主流架构。

架构特性：采用商用微处理器，每个节点配备独立私有局部内存；节点间通过**定制高带宽、低延迟专用互连网络**通信；支持上千至上万处理器大规模扩展。

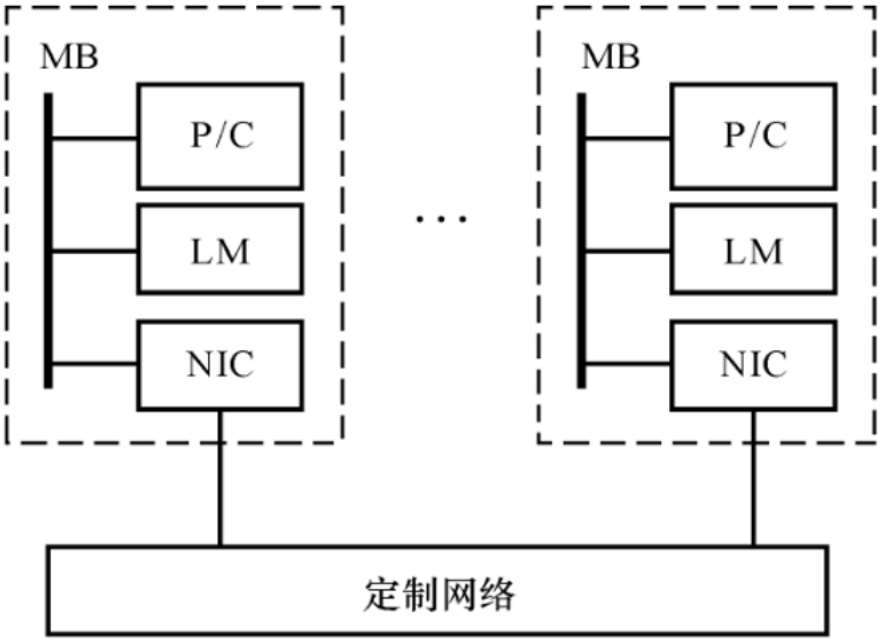
运行机制：异步MIMD架构，程序拆分为多个独立进程，每个进程占用私有地址空间，进程间通过**消息传递**实现交互。

应用场景：高端科学计算、工程模拟、信号处理、气象模拟、核能仿真等高强度计算场景

代表机型：Intel Paragon、Cray T3E、曙光1000、神威·太湖之光、2024年榜首El Capitan超算

即：**存储完全私有**：物理内存分布式、地址空间完全独立，没有全局统一逻辑地址

通信方式：异步 MIMD 架构，进程之间**必须通过消息传递**交换数据，无法直接远程访存



5. COW 工作站集群

核心定位：低成本、轻量化的变形MPP架构，依托通用设备搭建并行集群。

五大核心特征（必考区分点）：

节点完整：每个节点为独立无头工作站/PC/SMP主机，无需外设即可独立运行

低成本互连：采用以太网、FDDI、ATM等通用商用网络，无需定制硬件

本地磁盘：所有节点自带本地存储磁盘（MPP节点无本地磁盘）

松散耦合：网络接口挂载I/O总线，耦合度低（MPP接口挂载存储总线，紧耦合）

完整系统：每个节点运行完整UNIX操作系统，附加并行支撑软件层；MPP仅运行微核系统

代表机型：Berkeley NOW、Alpha Farm、Digital Truclster

发展现状：COW名称已淘汰，统一归入通用“集群（Cluster）”概念。

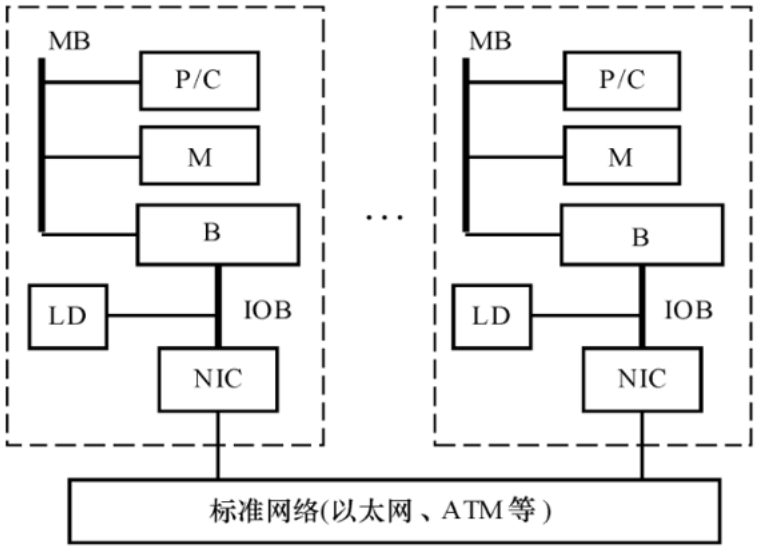
注意区别：

MPP： 网络接口挂载存储总线，紧耦合

网卡是系统的“核心部件”，通信路径短、速度快，节点之间“像一个整体”。

COW：网络接口挂载 I/O 总线，松散耦合

网卡是个普通外设，通信路径长、延迟高，节点之间“只是连起来了”。



5.1.5 MIMD系统存储架构分类（按寻址空间）

1. 单地址空间——共享存储架构

所有存储单元统一编址，全局唯一地址空间，所有处理器均可直接访问所有存储数据，分为两类：

UMA 均匀存储访问：物理存储集中，所有处理器访问任意存储单元的延迟、带宽完全一致，典型架构：PVP、SMP

NUMA 非均匀存储访问：物理存储分布式，逻辑统一地址；节点本地内存访问速度快，远程节点内存访问延迟高，典型架构：DSM

2. 多地址空间——非共享存储架构

各节点内存为私有资源，独立编址、独立地址空间，仅本地处理器可访问；跨节点数据交互必须通过消息传递完成，典型架构：MPP、COW。

5.2 多处理机Cache一致性

5.2.1 Cache一致性问题成因

多处理器系统中，共享数据会被加载到多个处理器的本地Cache中，形成多份数据副本。当**某一个处理器修改本地Cache副本数据后**，其余处理器Cache中的旧副本会与新数据不匹配，产生**数据不一致问题**，导致程序运行错误。

5.2.2 存储器一致性三大判定条件

满足以下三条规则，即可判定存储系统具备一致性：（如果对某个数据项的任何读操作均可得到其最新写入的值，则认为这个存储系统是一致的。）

自读写一致：处理器P对存储单元X写数据后，无其他处理器修改X，P再次读取X，一定读到自己刚写入的最新值。

跨处理器读写一致：处理器P写入X后，无其他修改操作，处理器Q读取X，一定读到P写入的最新值。

写操作串行化：任意两个处理器对同一存储单元的多次写操作，在所有处理器视角中，执行顺序完全一致，无乱序冲突。

5.2.3 多处理机Cache的两大核心功能

1. **共享数据迁移（适配写操作）**

将数据从数据拥有者节点，迁移到数据请求者的本地Cache中。核心作用：减少远程共享数据的访问延迟，降低共享存储器的带宽占用，提升写操作效率。

2. **共享数据复制（适配读操作）**

多个处理器同时读取同一共享数据时，允许所有节点Cache同时保留该数据副本。核心作用：避免多节点读请求冲突，大幅降低共享数据访问延迟。

5.2.4 两类Cache一致性协议总览

核心区别：跟踪共享数据块状态的方式不同，适配场景不同

监听式协议：基于总线广播，所有Cache监听总线消息，适合**小规模多处理机**，扩展性差、实现简单

目录式协议：基于专用目录记录数据状态，无广播操作，适合**大规模分布式多处理机**，扩展性强、逻辑复杂

5.2.5 两种写一致性策略

1. **写作废协议（主流常用）**

核心逻辑：处理器执行写操作前，通过总线广播作废所有其他节点的同名数据副本，确保自身拥有数据唯一访问权，再完成本地写操作。

优势：连续多次写同一数据时，仅需一次作废操作，后续写操作无需广播，效率高；按Cache块整体操作，开销可控。

2. 写更新协议

核心逻辑：处理器写数据时，广播新数据，实时更新所有节点Cache中的同名副本，所有节点数据始终同步。

优缺点：处理器读取其他节点写入数据的延迟极低；但多次连续写操作会产生多次广播，带宽开销大，按字/字节操作，开销精细但冗余多。

两者核心性能差异

连续无读多次写：写作废仅1次广播，写更新多次广播，写作废更优

单块多字连续写：写作废仅首次作废，写更新逐字广播，写作废更优

跨节点读取延迟：写更新数据实时同步，延迟远小于写作废

- 写更新：A 写完 → B 已经是新值 → 读 = 命中 → 延迟小
- 写作废：A 写完 → B 是无效 → 读 = 不命中 → 重新取 → 延迟大

写作废是针对Cache块进行操作，而写更新则是针对字（或字节）进行

- **监听 / 目录：解决“怎么找到其他 Cache”**（跟踪方式）
- **作废 / 更新：解决“找到后怎么处理数据”**（写策略）

5.2.6 Cache 与主存之间的写规则

- 写直达：写 Cache → 立刻写主存 → 简单、费带宽
- 写回法：写 Cache → 标记脏 → 替换时才写回 → 高效、省总线

5.3 监听式一致性协议（重点实操考点）

5.3.1 监听协议三大实现关键

硬件基础：处理器通过**可广播总线**互连，所有Cache控制器挂载总线

消息广播：本地Cache涉及全局操作时，抢占总线控制权，广播操作消息

全局监听：所有Cache实时监听总线，匹配自身Cache数据，触发对应操作

串行化保障：总线独占机制，天然实现写操作串行化，避免并发冲突

5.3.2 CPU访存的四种基础场景

本地命中（无需总线操作）：ReadHit（读命中）、WriteHit（写命中）

本地缺失（需总线广播）：ReadMiss（读缺失）、WriteMiss（写缺失）

5.3.3 总线核心消息与Cache配置

1. 三类总线消息

RdMiss：读不命中，向总线发送请求，调取远程/主存数据块

WtMiss：写不命中，广播写请求，调取远程/主存数据块，申请独占数据块

Invalidate：作废消息，仅作废其他节点副本，不调取数据块

2. Cache必备配置

Tag标识：用于总线监听匹配，快速判断本地是否存在目标数据

有效位：置0即可作废对应Cache块数据

共享位：1=多节点共享该块；0=本地独占该块

块拥有者：独占状态下，唯一持有最新数据副本的处理器

5.3.4 Cache块三状态定义（I/S/M）

I 无效态：当前Cache块数据失效，不可使用，无有效副本

S 共享态：多个节点持有该数据副本，所有副本与主存数据完全一致

M 已修改态：该块已经被修改过，并且还没写入存储器。（块中的内容是最新的，系统中唯一的最新副本）

5.3.5 完整状态转换操作表（必考）

1. 本地处理器请求触发操作

请求类型	当前块状态	核心操作	次态
ReadHit 读命中	S/M	直接读取本地Cache数据，无总线操作	保持原态
ReadMiss 读缺失	I/S	总线发送RdMiss，调取数据块至本地Cache	S

ReadMiss 读缺失	M	先将修改数据写回主存，再发送RdMiss调块	S
WriteHit 写命中	M	直接本地写数据，无总线操作	M
WriteHit 写命中	S	总线广播Invalidate，作废所有远程副本，本地写数据	M
WriteMiss 写缺失	I	总线发送WtMiss，调取数据并独占	M
WriteMiss 写缺失	S	发送WtMiss调块+Invalidate作废远程副本	M
WriteMiss 写缺失	M	写回旧数据、发送WtMiss、作废远程副本	M

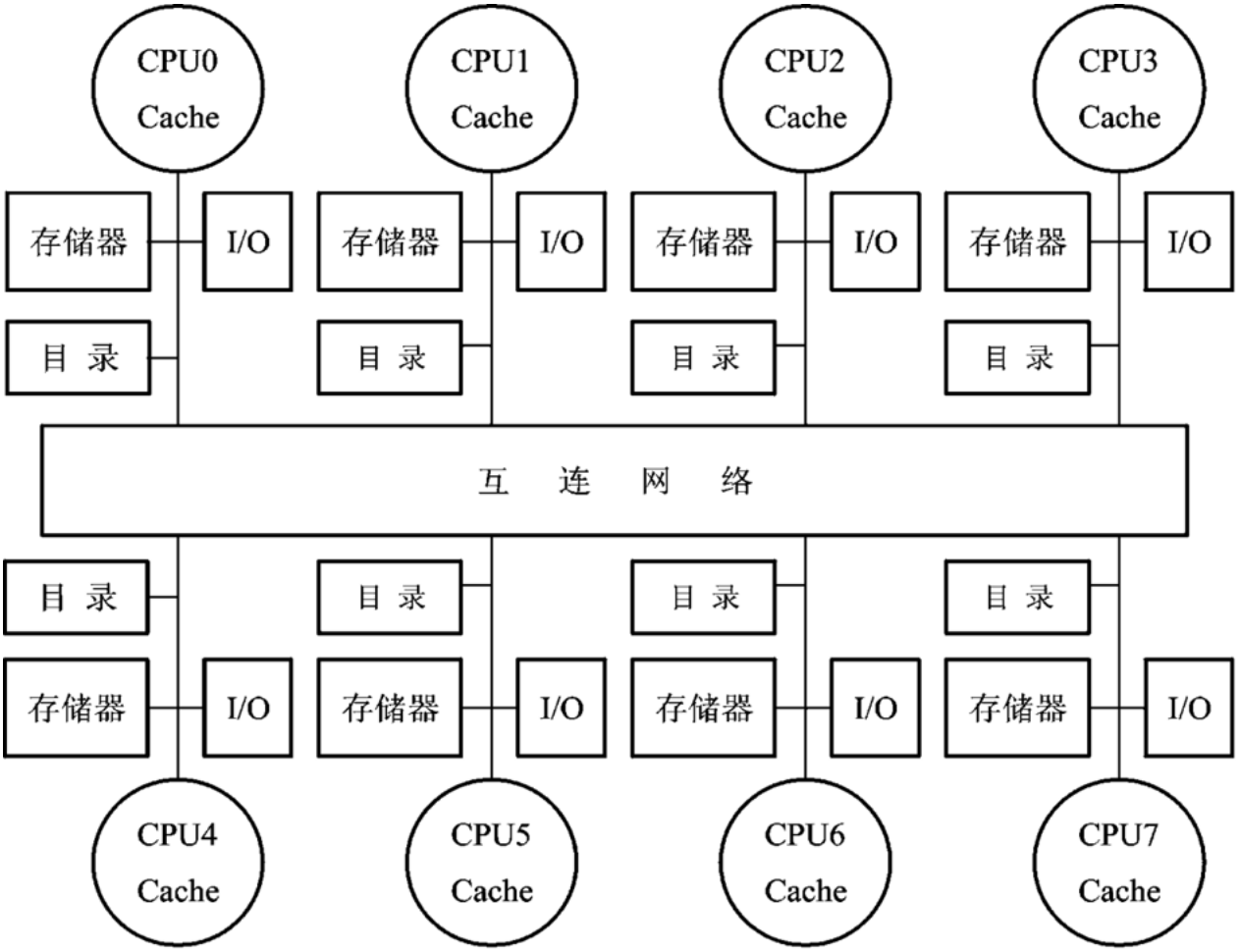
2. 总线远程请求触发操作

总线请求类型	当前块状态	核心操作	次态
RdMiss 远程读缺失	S	无操作，允许主存/其他节点提供数据	S
RdMiss 远程读缺失	M	本地修改数据写回主存，将数据共享至总线	S
WtMiss 远程写缺失	S	**尝试提供数据**，本地副本作废	I
WtMiss 远程写缺失	M	写回数据、**共享数据**、本地副本作废	I
Invalidate 远程作废	S	本地Cache对应数据块作废	I

5.4 目录式一致性协议（高分难点）

5.4.1 协议核心思想

监听协议依赖总线广播，节点数量增多后总线冲突严重、扩展性极差。目录协议通过**专用集中/分布式目录数据结构**，统一记录每一个存储块的状态、共享节点集合，所有一致性操作通过目录定向调度，**完全摒弃广播机制**，适配大规模多处理机系统。



对每个结点增加目录后的分布式存储器多处理机（DSM）

- 位向量：用 N 位二进制数，记录哪些 CPU 缓存了数据块。
- 目录开销：总信息量和处理器个数的平方（N^2）成正比

5.4.2 三大核心节点定义

本地节点：主动发出读写访存请求的计算节点

宿主节点：存储目标数据块物理内存、对应目录项的节点（地址高位对应节点编号）

远程节点：本地、宿主节点之外，持有目标数据块Cache副本的节点

备注：三类节点可重叠，本地节点可同时为宿主节点

> 在分布式共享存储系统里，虽然内存是分散在各个节点上的，但所有节点看到的是**同一个连续的地址空间**（就像单台电脑的内存一样）。

> 系统是这样拆分地址的：
>
> - **地址高位 = 节点地址**（决定数据存在哪个节点）
> - **地址低位 = 节点内偏移量**（决定数据在节点内存里的具体位置）
>
> 假设系统有 4 个节点（编号 0~3），地址是 32 位，那么：
>
> - 地址的**高 2 位**用来表示节点号：`00` →节点 0，`01` →节点 1，`10` →节点 2，`11` →节点 3
> - 剩下的**低 30 位**就是该节点内的内存偏移地址

5.4.3 存储块三状态定义（U/S/E）

U 未缓存态：所有处理器Cache中无该数据块副本，仅主存存储有效数据

S 共享态：一个或多个节点持有副本，**所有副本与主存数据一致**，支持多节点读

E 独占态：**仅一个节点持有修改后的最新副本，主存数据过时**，该节点为数据拥有者

5.4.4 本地节点动作解析

本地节点（Cache）分析

本地Cache	现态	自己搞定	读新	写新	废旧	写回并废旧	让宿主废别人	次态
ReadHit	S	√						S
ReadHit	M	√						M
ReadMiss	I		√					S
ReadMiss	S		√		√			S
ReadMiss	M		√			√		S
WriteHit	S						√	M
WriteHit	M	√						M
WriteMiss	I			√				M
WriteMiss	S			√	√			M
WriteMiss	M			√		√		M

操作解释：

- 读新：本地节点想要读一块新的。
- 写新：本地节点想要写一块新的。
- 废旧：本地节点告诉宿主我不要这块了，但是不需要写回。
- 写回并废旧：本地节点告诉宿主我不要这块了，但是需要写回。
- 让宿主废别人：让宿主把别人的块废了。

具体操作解释：

本地结点发给宿主结点（目录）的消息定义

- 读新 -- RdMiss（P，K）

  处理机P读取地址为K的数据时不命中，请求宿主结点提供数据（块），并要求把P加入共享集。

- 写新 -- WtMiss（P，K）

  处理机P对地址K进行写入时不命中，请求宿主结点提供数据，并使P成为所访问数据块的独占者。

- 废旧 -- MdSharer（P，K）

  用于当本地Cache中需要替换一个包含地址K的块、且该块未被修改过的情况。这个消息发给该块的宿主结点，请求它将P从共享集中删除。如果删除后共享集变为空集，则宿主结点还要将该块的状态改变为“未缓存”（U）。

- 写回并废旧 -- WtBack2（P，K）

  用于当本地Cache中需要替换一个包含地址K的块、且该块已被修改过的情况。这个消息发给该块的宿主结点，完成两步操作：①把该块写回；②进行与MdSharer相同的操作。

- 让宿主废别人 -- Invalidate（K）

  请求向所有拥有相应数据块副本（包含地址K）的远程Cache发Invalidate消息，作废这些副本。

说明：括号中的内容表示所带参数。
P：发出请求的处理机编号
K：所要访问的地址

5.4.5 宿主节点动作解析

共享集合：位向量记录拥有其副本的处理器的集合

宿主节点分析

现状	收到 本地节点消息	发消息 给远程节点	更新 共享集合	回应消息 给本地节点	次态
U	RdMiss(P, K)	无	{ }+{P}	DReply(D)	S
	WtMiss(P, K)	无	{ }+{P}	DReply(D)	E
	MdSharer(P, K)				
	WtBack2(P, K)				
	Invalidate(K)				
S	RdMiss(P, K)	无	{含K共享集}+{P}	DReply(D)	S
	WtMiss(P, K)	Invalidate(K)	{P}	DReply(D)	E
	MdSharer(P, K)	无	{含K共享集}-{P}		S 或 U
	WtBack2(P, K)				
	Invalidate(K)	Invalidate(K)	{P}		E
E	RdMiss(P, K)	Fetch (K)	{只有远程节点}+{P}	DReply(D)	S
	WtMiss(P, K)	Fetch&Inv (K)	{P}	DReply(D)	E
	MdSharer(P, K)				
	WtBack2(P, K)		{ }		U
	Invalidate(K)				

- Invalidate (K) 作废远程Cache中包含地址K的数据块。
- Fetch (K) 从远程Cache中取出包含地址K的数据块，并将之送到宿主结点。把远程Cache中那个块的状态改为“共享”。
- Fetch&Inv (K) 从远程Cache中取出包含地址K的数据块，并将之送到宿主结点。然后作废远程Cache中的那个块。
- DReply (D) 宿主节点将存储器中的数据返回给本地Cache。

5.4.6 远程节点动作解析

远程节点分析（Cache）

现状	收到 宿主节点消息	回应消息 给宿主节点	次态
S	Invalidate(K)		I
	Fetch(K)		
	Fetch&Inv(K)		
M	Invalidate(K)		
	Fetch(K)	WtBack（K，D）	S
	Fetch&Inv(K)	WtBack（K，D）	I

- WtBack（K,D） 把远程Cache中包含地址K的数据块写回到宿主结点中， 该消息是远程结点对宿主结点发来的“Fetch(K)”或“Fetch&Inv(K)”消息的响应。

5.4.7 全网核心消息定义（必考）

消息名	发送方	接收方	核心功能
RdMiss(P,K)	本地Cache	宿主目录	P节点读K地址缺失，请求数据，加入共享集
WtMiss(P,K)	本地Cache	宿主目录	P节点写K地址缺失，申请独占数据块
MdSharer(P,K)	本地Cache	宿主目录	无修改数据块替换，将P从共享集移除
WtBack2(P,K)	本地Cache	宿主目录	有修改数据块替换，写回数据+移除共享集
Invalidate(K)	宿主目录	远程Cache	作废所有远程节点K地址数据副本
Fetch(K)	宿主目录	远程Cache	取回K块数据，远程节点状态改为共享
Fetch&Inv(K)	宿主目录	远程Cache	取回K块数据，同时作废远程副本
DReply(D)	宿主目录	本地Cache	宿主向本地节点返回目标数据D
WtBack(K,D)	远程Cache	宿主目录	远程节点向宿主写回最新数据

5.4.6 三类节点完整操作逻辑

1. 本地Cache节点状态转换规则

核心逻辑：根据当前I/S/M状态，响应CPU读写请求，向宿主发送对应消息，切换状态。读操作最终趋向S态，写操作最终趋向M态。

2. 宿主目录节点状态转换规则

核心逻辑：接收本地节点消息，调度远程节点操作、更新共享集、切换U/S/E状态、回复本地节点。独占态访问需先取回远程数据，共享态写操作需先作废所有远程副本。

3. 远程Cache节点响应规则

接收宿主节点调度消息，完成数据写回、副本作废、状态切换，无主动发起权限，仅被动响应。

5.4.7 目录的三种核心结构（必考对比）

1. 全映像目录

结构：每个目录项配备**N位位向量**（N=处理器总数），每一位对应一个处理器，1=存在副本，0=无副本。

优点：逻辑简单、查询速度快、操作直接

缺点：空间开销极大，存储空间与处理器数量N^2成正比，处理器越多，开销爆炸式增长，可扩展性极差，仅适配小规模系统。

2. 有限映像目录

结构：**固定每个数据块的最大副本数量m**，目录项位数固定为$m \times \log_2 N$，不随处理器总数扩容。

优点：大幅降低存储空间开销，可扩展性优于全映像目录

缺点：存在副本上限，当所需副本数超过m时，必须驱逐已有副本，产生额外替换开销。

3. 链式目录

结构：用**动态指针链表**记录共享节点集合，副本增加则链表扩容，副本减少则链表收缩。分为单链、双链两种实现方式。

优点：无副本数量限制，完美适配大规模节点扩展，可扩展性最优

缺点：单链替换需遍历链表，查询效率低；双链效率高但协议逻辑复杂、硬件开销翻倍。

实现方式	删除操作时间	存储开销	一致性复杂度	适用场景
单链法 – 修改指针	O (n)（遍历找前驱）	低（单指针）	简单	小规模系统

单链法 – 直接作废	O (1)（但会误杀）	低（单指针）	最简单	对性能不敏感的场景
双链法	O (1)（直接删）	高（双指针）	复杂	大规模、高性能系统